

Physics-Informed Neural Networks for the Dual-Phase-Lag Bioheat Equation

A Comparative Study of PINN, SA-PINN, gPINN, and SA-gPINN
with Validation against Crank-Nicolson and Newmark- β Schemes

Pulkit Garg

Department of Mathematical Sciences, IIT (BHU) Varanasi

Roll No.: 22124033 Supervised by: Dr. Santwana Mukhopadhyay

Abstract

This work presents a systematic comparative study of four Physics-Informed Neural Network (PINN) variants applied to the one-dimensional Dual-Phase-Lag (DPL) bioheat transfer equation, which models the non-Fourier propagation of heat in biological tissue during hyperthermia cancer treatment. The four variants studied are: (i) the standard PINN [?]; (ii) the Self-Adaptive PINN (SA-PINN) [2], which uses adversarially-trained loss weights; (iii) the Gradient-Enhanced PINN (gPINN) [3], which augments the loss with gradient residuals of the PDE; and (iv) a novel hybrid *SA-gPINN*, combining self-adaptive weighting with gradient enhancement. All methods use the same fully-connected neural network architecture and a two-stage optimisation strategy consisting of Adam followed by L-BFGS-B fine-tuning.

A key methodological contribution is the trial solution $\theta(x, F_o) = F_o^2 \mathcal{N}(x, F_o; \mathbf{w})$, which exactly satisfies both homogeneous initial conditions by construction, eliminating the IC loss term from optimisation. Since the full DPL equation has no closed-form analytical solution, two independent classical schemes — Crank-Nicolson (first-order state-space formulation) and Newmark- β (second-order structural dynamics formulation) — are implemented as reference solvers, agreeing to machine precision (1.17×10^{-12}).

Quantitative comparison is performed using a three-metric framework: PDE residual RMS, boundary-condition residuals, and absolute difference from the numerical reference solution. **SA-PINN ranks first across all three metrics**, achieving PDE residual 1.83×10^{-3} and maximum reference difference 1.21×10^{-3} . Although SA-gPINN combines two advanced techniques, its performance is degraded by the sharp Gaussian tumour source ($a = 200$), which generates large $\partial P_r / \partial x$ terms that corrupt the gradient-residual loss $\mathcal{L}_g$ and destabilise the three-weight adversarial optimisation process. While classical numerical schemes remain faster and more accurate for this one-dimensional forward problem, PINNs offer important advantages for inverse problems, parameter estimation, and extension to complex three-dimensional patient-specific geometries where conforming mesh generation becomes difficult or impractical.

Contents

1	Introduction	4
1.1	Motivation: Hyperthermia Cancer Treatment	4
1.2	The Computational Challenge	4
1.3	Physics-Informed Neural Networks	5
1.4	Specific Contributions of This Work	5
2	Mathematical Formulation	6
2.1	Governing Dimensionless DPL Bioheat Equation	6
2.2	Heat Source Modelling the Tumour Region	6
2.3	Initial and Boundary Conditions	7
2.4	Reduction to Limiting Cases	7
2.5	Vector-Matrix Form for Numerical Schemes	7
3	Neural Network Architecture and Trial Solution	8
3.1	Architecture	8
3.2	The Trial Solution: Hard-Coding the Initial Conditions	8
4	The Four PINN Variants: Loss Functions and Training	9
4.1	The PDE Residual	10
4.2	Method 1: Standard PINN	10
4.3	Method 2: Self-Adaptive PINN (SA-PINN)	10
4.4	Method 3: Gradient-Enhanced PINN (gPINN)	11
4.5	Method 4: SA-gPINN (Proposed Combination)	11
4.6	Structural Comparison of the Four Methods	12
5	Training Strategy and Computational Cost	12
5.1	Two-Phase Optimisation: Adam Followed by L-BFGS-B	12
5.2	Sampling Strategy	12
5.3	Hyperparameter Summary	13
5.4	Computational Cost: Order-of-Autodiff Analysis	13
6	Classical Numerical Reference Schemes	14
6.1	Crank-Nicolson Finite-Difference Scheme	14
6.1.1	Step 1: Spatial Discretisation and Boundary Treatment	14
6.1.2	Step 2: First-Order State-Space Reduction	14
6.1.3	Step 3: Crank-Nicolson Integration	15
6.2	Newmark- β Time-Integration Scheme	15

6.2.1	Step 1: Newmark Update Formulas	15
6.2.2	Step 2: Solving for the New Displacement	15
6.2.3	Step 3: Initial Acceleration and Time-Stepping Loop	16
6.3	Validation by Cross-Scheme Agreement	16
7	Validation Strategy: The Three-Metric Framework	16
7.1	Why a Single Metric is Insufficient	16
7.2	Metric 1: PDE Residual RMS on Native Grid (PINN-only)	17
7.3	Metric 2: BC Residuals (Comparable Across All Six Methods)	17
7.4	Metric 3: Solution Difference from CN Reference	17
8	Results	17
8.1	Visual Inspection of PINN Solutions	17
8.2	Six-Method Solution Comparison	18
8.3	PDE Residual Maps and Metric 1	18
8.4	BC Residuals and Metric 2	19
8.5	Solution Difference from CN Reference: Metric 3	19
8.6	Pairwise Differences Among All Six Methods	20
8.7	Composite Ranking: SA-PINN Wins on All Three Metrics	20
9	Discussion	20
9.1	Why CN and Newmark Are References, Not Competitors	20
9.2	Why SA-PINN Ranked First Among PINN Methods	21
9.3	Why SA-gPINN Underperformed Relative to SA-PINN	21
9.4	When Would SA-gPINN Outperform SA-PINN?	22
9.5	Why Use PINNs At All If Classical Schemes Are Faster and More Accurate?	22
9.6	Limitations and Future Scope	23
10	Conclusion	24
	Appendix: Figures	26

1 Introduction

1.1 Motivation: Hyperthermia Cancer Treatment

Hyperthermia is the controlled elevation of tissue temperature to the range 41°C–46°C for therapeutic purposes. It is widely used as an adjuvant therapy alongside chemotherapy and radiotherapy, since elevated temperatures sensitise cancer cells to cytotoxic agents while sparing healthy tissue [1]. Successful clinical deployment of hyperthermia requires *predictive mathematical models* that accurately compute the spatio-temporal temperature distribution inside the tumour and surrounding tissue, given the heat source profile (laser, microwave, ultrasound) and the patient’s tissue thermophysical properties.

The classical Pennes bioheat equation [?], derived from Fourier’s law of heat conduction, assumes *infinite* thermal propagation speed — a perturbation at any point of the domain influences all other points instantaneously. While acceptable for steady-state or slowly-varying problems, this assumption is physically unrealistic in the highly non-homogeneous micro-structure of biological tissue, where heat conduction is mediated by complex cell-vessel-extracellular-fluid interactions [?]. To address this limitation, Tzou [?] proposed the **Dual-Phase-Lag** (DPL) model, which generalises Fourier’s law to:

$$\mathbf{q}(x, t + \tau_q) = -k \nabla T(x, t + \tau_T) \quad (1)$$

where $\mathbf{q}$ is the heat flux, T is temperature, k is the conductivity, and $\tau_q, \tau_T \geq 0$ are characteristic time lags representing the delay of heat flux response to a temperature gradient (τ_q) and the delay of temperature gradient response to a heat flux (τ_T). Expanding both sides to first order in their respective phase lags and combining with the energy balance equation yields the DPL bioheat equation, which captures the wave-like propagation of thermal disturbances in soft tissue and reduces to Pennes’ model when $\tau_q = \tau_T = 0$, and to the Cattaneo-Vernotte thermal-wave model when $\tau_T = 0$.

1.2 The Computational Challenge

Conventional numerical methods for solving the DPL equation include finite-difference, finite-element, wavelet-Galerkin, and boundary-element methods [1]. All such methods discretise space and time on a fixed mesh and propagate the solution forward via stable time-stepping. While these methods are mature and computationally fast for low-dimensional forward problems, they have three fundamental limitations:

1. **Inverse problems are expensive.** Recovering unknown tissue parameters such as τ_q and τ_T from temperature measurements requires repeated full re-solves inside an outer optimisation loop — one re-solve per parameter evaluation.
2. **Parametric studies require many runs.** Evaluating temperature distribution for N different tumour locations, source amplitudes, or perfusion coefficients requires N independent simulations.
3. **3D patient-specific geometry requires conforming meshes.** Generating

tetrahedral meshes that align with irregular tumour boundaries reconstructed from CT/MRI scans is a labour-intensive and error-prone preprocessing step.

1.3 Physics-Informed Neural Networks

Physics-Informed Neural Networks (PINNs), introduced by Raissi, Perdikaris, and Karniadakis [?], address all three limitations simultaneously. A PINN parameterises the solution of a PDE as a deep neural network $\theta(x, F_o; \mathbf{w})$ and trains the parameters $\mathbf{w}$ by directly minimising a composite physics-based loss functional comprising the PDE residual, boundary conditions, and initial conditions. The key insight is that all derivatives appearing in the PDE can be computed *exactly* via automatic differentiation through the neural network graph — no finite-difference approximation is needed and no mesh is required.

The original PINN of Raissi et al. has spawned a rich family of variants addressing specific failure modes:

- **SA-PINN** [2]: introduces *learnable adversarial weights* on individual loss terms, dynamically balancing PDE residual against boundary/initial condition satisfaction during training.
- **gPINN** [3]: augments the standard loss with gradients of the PDE residual, exploiting the fact that the *exact* solution satisfies $\partial R/\partial x = 0$ and $\partial R/\partial F_o = 0$ in addition to $R = 0$.

1.4 Specific Contributions of This Work

Despite the rich literature on PINN variants, no prior comparative study has applied them systematically to the DPL bioheat equation. The present work makes the following specific contributions:

1. **Implementation of four PINN variants for the full DPL bioheat equation.** We implement standard PINN, SA-PINN, gPINN, and a novel SA-gPINN combining the latter two, on the one-dimensional dimensionless DPL equation with Gaussian tumour source.
2. **Trial solution that exactly satisfies the initial conditions.** We use $\theta = F_o^2 \mathcal{N}$, which by direct algebraic verification satisfies both $\theta(x, 0) = 0$ and $\partial\theta/\partial F_o|_{F_o=0} = 0$ for any network $\mathcal{N}$. This eliminates the IC loss term and reduces the optimisation to two competing objectives.
3. **SA-gPINN architecture.** A direct extension of SA-PINN that adds an adversarial weight on the gradient-residual loss term, enabling automatic balancing of three competing physics constraints.
4. **Independent dual-scheme reference validation.** We implement Crank-Nicolson (state-space first-order) and Newmark- β (structural-dynamics second-order) classical schemes, which agree to 1.17×10^{-12} , establishing a trusted ground truth despite the absence of a closed-form solution.

5. **Three-metric comparison framework.** A fair ranking procedure based on (a) PDE residual on native grid for PINN-only comparison, (b) boundary-condition residuals comparable across all six methods, and (c) absolute solution difference from the trusted reference.
6. **Mathematical analysis of why SA-PINN beats SA-gPINN on this problem.** We trace the underperformance of SA-gPINN to a specific mechanism: the sharp Gaussian source generates a large $\partial P_r / \partial x$ which dominates the gradient residual loss $\mathcal{L}_g$ and destabilises the three-weight adversarial game.

2 Mathematical Formulation

2.1 Governing Dimensionless DPL Bioheat Equation

The dimensional DPL bioheat equation in tissue, accounting for blood perfusion (Pennes term) and metabolic generation, reads [1, ?]:

$$\tau_q \rho c \frac{\partial^2 T}{\partial t^2} + (\rho c + \tau_q \rho_b c_b \omega_b) \frac{\partial T}{\partial t} + \rho_b c_b \omega_b T - k \frac{\partial^2 T}{\partial X^2} - k \tau_T \frac{\partial^3 T}{\partial X^2 \partial t} = \rho_b c_b \omega_b T_b + Q_m + Q_r(X), \quad (2)$$

on $X \in [0, L]$, $t \in [0, t_f]$, where $T(X, t)$ is the tissue temperature, T_b is the arterial blood temperature, ω_b is the blood perfusion rate, Q_m is metabolic heat generation, and $Q_r(X)$ is the externally applied heating power profile.

Introducing dimensionless variables $x = X/L$, $F_o = \alpha t/L^2$, $\theta = (T - T_0)/(T_b - T_0)$, where $\alpha = k/(\rho c)$ is the thermal diffusivity, equation (2) reduces to the dimensionless form used throughout this work:
$$F_{oq} \frac{\partial^2 \theta}{\partial F_o^2} + \left(1 + F_{oq} P_f^2\right) \frac{\partial \theta}{\partial F_o} + P_f^2 \theta - \frac{\partial^2 \theta}{\partial x^2} - F_{oT} \frac{\partial^2 \theta}{\partial x \partial F_o} = P_f^2 \theta_b + P_m + P_r(x), \text{ on } x \in [0, 1], F_o \in [0, 1], \text{ where:}$$

Table 1: Dimensionless parameters of the DPL bioheat equation.

Parameter	Symbol	Value	Physical meaning
Phase lag of heat flux	$F_{oq} = \alpha \tau_q / L^2$	0.5	Wave-like memory of $\mathbf{q}$
Phase lag of temp. gradient	$F_{oT} = \alpha \tau_T / L^2$	0.01	Diffusive memory of ∇T
Blood perfusion coefficient	$P_f = L \sqrt{\rho_b c_b \omega_b / k}$	0.1	Pennes perfusion strength
Blood (arterial) temperature	θ_b	0.0	Reference base level
Metabolic heat generation	$P_m = L^2 Q_m / [k(T_b - T_0)]$	0.0	Background heat
Source amplitude	P_{r0}	1.0	Peak heating intensity
Gaussian sharpness	a	200	Source spatial concentration
Tumour location	x^*	0.5	Centre of domain
Robin BC coefficients	A, B	1.0	Mixed BC weights

2.2 Heat Source Modelling the Tumour Region

The Gaussian heat source models a localised tumour region with externally applied focal heating (e.g. focused-ultrasound or laser irradiation):

$$P_r(x) = P_{r0} \exp(-a(x - x^*)^2), \quad P_{r0} = 1, \quad x^* = 0.5, \quad a = 200. \quad (3)$$

The choice $a = 200$ produces a sharp Gaussian with a characteristic width $\sigma = 1/\sqrt{2a} \approx 0.05$, which is one twentieth of the domain length. As we will analyse later, this sharpness is the source of the difficulty experienced by gradient-enhanced methods.

2.3 Initial and Boundary Conditions

Initial conditions (homogeneous, both temperature and its time rate vanish at $F_o = 0$):

$$\theta(x, 0) = 0, \quad \left. \frac{\partial \theta}{\partial F_o} \right|_{F_o=0} = 0 \quad \forall x \in [0, 1]. \quad (4)$$

The second IC is required because the DPL equation is *second order in time* (due to the $F_{oq} \theta_{F_o F_o}$ term), so two initial conditions are needed for a well-posed problem.

Boundary conditions:

$$\left. \frac{\partial \theta}{\partial x} \right|_{x=0} = 0 \quad (\text{Neumann — symmetry / insulation}), \quad (5)$$

$$A \left. \frac{\partial \theta}{\partial x} \right|_{x=1} + B \theta(1, F_o) = 0 \quad (\text{Robin / third-kind — convective skin surface}). \quad (6)$$

The Neumann BC at $x = 0$ models a symmetric tumour at the centre of the tissue domain. The Robin BC at $x = 1$ models heat exchange with the ambient environment at the skin surface (Newton's law of cooling).

2.4 Reduction to Limiting Cases

The DPL equation (2.1) contains several classical bioheat models as special cases:

- Setting $F_{oq} = F_{oT} = 0$ recovers the **Pennes equation** (parabolic, infinite signal speed).
- Setting $F_{oT} = 0$ recovers the **thermal wave / Cattaneo-Vernotte model** (hyperbolic, finite signal speed but no spatial memory).
- Setting $F_{oq} = F_{oT}$ recovers the **single-phase-lag** model.

The full DPL model with $F_{oq} \neq F_{oT} \neq 0$ has both wave-like and diffusive features, and is the subject of this study.

2.5 Vector-Matrix Form for Numerical Schemes

After spatial discretisation by central finite differences on the grid $x_i = (i-1)h$, $h = 1/(N_x - 1)$ for $i = 1, \dots, N_x$, the DPL equation (2.1) reduces to the second-order linear ODE system:

$$\boxed{\mathbf{M} \ddot{\boldsymbol{\theta}}(F_o) + \mathbf{C} \dot{\boldsymbol{\theta}}(F_o) + \mathbf{K} \boldsymbol{\theta}(F_o) = \mathbf{N}(F_o)} \quad (7)$$

where $\boldsymbol{\theta}(F_o) \in \mathbb{R}^{N_x}$ is the discrete temperature vector, $(\dot{\cdot}) = d/dF_o$, and:

$$\mathbf{M} = F_{oq} \mathbf{I}, \quad (8)$$

$$\mathbf{C} = (1 + F_{oq} P_f^2) \mathbf{I} - F_{oT} \mathbf{M}_1, \quad (9)$$

$$\mathbf{K} = P_f^2 \mathbf{I} - \mathbf{M}_1, \quad (10)$$

$$\mathbf{N}(F_o) = [P_f^2 \theta_b + P_m + P_r(x_i)]_{i=1}^{N_x}. \quad (11)$$

The matrix $\mathbf{M}_1$ is the discrete second-derivative operator $\partial^2/\partial x^2$ incorporating the boundary conditions (5)–(6); its construction is given in Section 6. This vector-matrix form is the starting point for both Crank-Nicolson and Newmark- β schemes.

3 Neural Network Architecture and Trial Solution

3.1 Architecture

All four PINN variants share an identical fully-connected feedforward network with tanh activations. The architecture is summarised in Table 2.

Table 2: Neural network architecture (identical for all four PINN methods).

Component	Specification
Architecture vector	[2, 80, 80, 80, 80, 1]
Input neurons	2 — spatial coordinate x and time F_o
Hidden layers	4 layers of 80 neurons each
Activation function	$\tanh(\cdot)$ at all hidden layers
Output	Single scalar $\mathcal{N}(x, F_o; \mathbf{w})$
Weight initialisation	Xavier/Glorot uniform
Input normalisation	(x, F_o) mapped to $[-1, 1]$ by affine scaling
Total trainable parameters	$\approx 26,000$

The network defines a continuous function $\mathcal{N} : [-1, 1]^2 \rightarrow \mathbb{R}$ as the composition:

$$\mathcal{N}(x, F_o; \mathbf{w}) = \mathbf{W}^{(L)} \sigma(\cdots \sigma(\mathbf{W}^{(1)} \mathbf{z}_0 + \mathbf{b}^{(1)})) + \mathbf{b}^{(L)}, \quad \sigma(z) = \tanh(z), \quad (12)$$

where $\mathbf{z}_0 = [2x - 1, 2F_o/F_{o\max} - 1]^\top$ is the normalised input and $\mathbf{w} = \{\mathbf{W}^{(\ell)}, \mathbf{b}^{(\ell)}\}$ collects all trainable weights and biases. The choice of tanh activation guarantees that derivatives of all orders exist and are continuous, which is essential for PINN training (the loss requires exact second-order derivatives, and gPINN requires third-order).

3.2 The Trial Solution: Hard-Coding the Initial Conditions

A central methodological choice in this work is the use of the **trial solution** (also called *ansatz* in the literature):

$$\boxed{\theta(x, F_o; \mathbf{w}) = F_o^2 \cdot \mathcal{N}(x, F_o; \mathbf{w}).} \quad (13)$$

This is not the network output directly, but the network output multiplied by F_o^2 . The choice is motivated by the following algebraic property.

[IC Hard-Coding] For any network output $\mathcal{N} : [-1, 1]^2 \rightarrow \mathbb{R}$ that is differentiable in F_o , the trial solution (13) satisfies both initial conditions (4) *exactly*, regardless of the parameters $\mathbf{w}$:

1. $\theta(x, 0; \mathbf{w}) = 0$,
2. $\frac{\partial \theta}{\partial F_o}(x, 0; \mathbf{w}) = 0$.

Proof. Part (1). At $F_o = 0$:

$$\theta(x, 0) = 0^2 \cdot \mathcal{N}(x, 0; \mathbf{w}) = 0 \cdot \mathcal{N}(x, 0) = 0.$$

Part (2). Differentiating (13) in F_o via the product rule:

$$\frac{\partial \theta}{\partial F_o} = 2F_o \mathcal{N}(x, F_o; \mathbf{w}) + F_o^2 \frac{\partial \mathcal{N}}{\partial F_o}(x, F_o; \mathbf{w}).$$

Evaluating at $F_o = 0$:

$$\left. \frac{\partial \theta}{\partial F_o} \right|_{F_o=0} = \underbrace{2 \cdot 0}_{=0} \cdot \mathcal{N}(x, 0) + \underbrace{0^2}_{=0} \cdot \frac{\partial \mathcal{N}}{\partial F_o}(x, 0) = 0 + 0 = 0.$$

Both ICs hold *identically*, regardless of $\mathbf{w}$ or the network architecture. □ □

Why this matters. A standard PINN formulation includes an IC loss term $\mathcal{L}_{\text{IC}}$ in addition to the PDE residual $\mathcal{L}_r$ and BC loss $\mathcal{L}_{\text{BC}}$:

$$\mathcal{L}_{\text{standard}} = \mathcal{L}_r + \mathcal{L}_{\text{BC}} + \mathcal{L}_{\text{IC}}.$$

With the trial solution (13), $\mathcal{L}_{\text{IC}} = 0$ identically. The network never has to “learn” the initial condition; it is built into the parametrisation. Consequently:

- The total loss reduces to $\mathcal{L} = \mathcal{L}_r + \mathcal{L}_{\text{BC}}$ only.
- The optimiser never spends gradient updates on satisfying the IC — those gradient updates can be redirected to $\mathcal{L}_r$ and $\mathcal{L}_{\text{BC}}$.
- Adversarial weighting schemes (SA-PINN, SA-gPINN) become better-conditioned because the loss landscape is reduced from three competing terms to two (or three for gPINN/SA-gPINN with the gradient term).

The boundary conditions (5)–(6) cannot be similarly hard-coded by a simple multiplicative ansatz, because the Robin BC (6) mixes the function value θ with its derivative $\partial\theta/\partial x$ — such combinations cannot be eliminated by a separable factor. The boundary conditions are therefore enforced through the loss term $\mathcal{L}_{\text{BC}}$ in all four methods.

4 The Four PINN Variants: Loss Functions and Training

This section presents in detail the four PINN methods compared in this work. All four use the trial solution (13) and the same network architecture (Table 2). They differ only in the structure of the loss functional and the training scheme.

4.1 The PDE Residual

For all methods, the *pointwise* PDE residual at any point $(x, F_o) \in [0, 1]^2$ is:

$$R(x, F_o; \mathbf{w}) = F_{oq} \frac{\partial^2 \theta}{\partial F_o^2} + (1 + F_{oq} P_f^2) \frac{\partial \theta}{\partial F_o} + P_f^2 \theta - \frac{\partial^2 \theta}{\partial x^2} - F_{oT} \frac{\partial^2 \theta}{\partial x \partial F_o} - (P_f^2 \theta_b + P_m + P_r(x)), \quad (14)$$

where $\theta = F_o^2 \mathcal{N}$ is the trial solution and all derivatives of θ are computed through it. The exact solution satisfies $R(x, F_o) \equiv 0$ everywhere; PINN training drives R towards zero in a least-squares sense over a sample of collocation points. All derivatives in (14) are computed by automatic differentiation through the network graph (TensorFlow's `tf.gradients`), *not* by finite differences.

4.2 Method 1: Standard PINN

The standard PINN loss is the unweighted sum of two terms:

$$\mathcal{L}_{\text{PINN}}(\mathbf{w}) = \mathcal{L}_r(\mathbf{w}) + \mathcal{L}_{\text{BC}}(\mathbf{w}), \quad (15)$$

where the PDE residual loss $\mathcal{L}_r$ averages R^2 over N_r randomly sampled collocation points:

$$\mathcal{L}_r(\mathbf{w}) = \frac{1}{N_r} \sum_{i=1}^{N_r} R(x_i, F_{oi}; \mathbf{w})^2, \quad (x_i, F_{oi}) \sim \text{Unif}([0, 1]^2), \quad (16)$$

and the boundary loss $\mathcal{L}_{\text{BC}}$ averages the squared BC residual over N_b randomly sampled boundary points:

$$\mathcal{L}_{\text{BC}}(\mathbf{w}) = \frac{1}{N_b} \sum_{j=1}^{N_b} \left[\left(\frac{\partial \theta}{\partial x} \Big|_{x=0, F_{oj}} \right)^2 + \left(A \frac{\partial \theta}{\partial x} \Big|_{x=1, F_{oj}} + B \theta(1, F_{oj}) \right)^2 \right], \quad (17)$$

with $F_{oj} \sim \text{Unif}([0, 1])$. Training minimises (15) jointly with respect to $\mathbf{w}$.

4.3 Method 2: Self-Adaptive PINN (SA-PINN)

The SA-PINN of McClenny and Braga-Neto [2] introduces two **learnable adversarial weights** $\lambda_r, \lambda_{\text{BC}} > 0$:

$$\mathcal{L}_{\text{SA-PINN}}(\mathbf{w}, \lambda_r, \lambda_{\text{BC}}) = \lambda_r \mathcal{L}_r(\mathbf{w}) + \lambda_{\text{BC}} \mathcal{L}_{\text{BC}}(\mathbf{w}). \quad (18)$$

Positivity is enforced by the softplus reparameterisation $\lambda = \ln(1 + e^{\tilde{\lambda}})$ with unconstrained logit $\tilde{\lambda} \in \mathbb{R}$.

Adversarial training. The key idea is that the network parameters $\mathbf{w}$ *minimise* the loss while the weights $\boldsymbol{\lambda}$ *maximise* it:

$$\mathbf{w} \leftarrow \mathbf{w} - \eta_w \nabla_{\mathbf{w}} \mathcal{L}, \quad \eta_w = 10^{-3}, \quad (19)$$

$$\tilde{\lambda}_k \leftarrow \tilde{\lambda}_k + \eta_{\lambda} \nabla_{\tilde{\lambda}_k} \mathcal{L}, \quad k \in \{r, \text{BC}\}, \quad \eta_{\lambda} = 10^{-2}. \quad (20)$$

This is a saddle-point optimisation: at the equilibrium, $\nabla_{\mathbf{w}} \mathcal{L} = 0$ and $\nabla_{\boldsymbol{\lambda}} \mathcal{L} = 0$. The maximisation step automatically allocates more weight to whichever loss term is currently *largest* (because that term has the steepest gradient with respect to its weight). Loss terms that are already small receive small weights; loss terms that are large are amplified, forcing the network to focus on them. This is the *self-adaptive* property.

For our DPL problem with the trial solution eliminating $\mathcal{L}_{\text{IC}}$, the SA-PINN game has only two loss terms ($\mathcal{L}_r$ vs. $\mathcal{L}_{\text{BC}}$) and two weights (λ_r vs. λ_{BC}), making the saddle-point well-conditioned and convergent.

4.4 Method 3: Gradient-Enhanced PINN (gPINN)

The gPINN of Yu et al. [3] introduces a gradient-residual loss term, leveraging a key mathematical observation: if the exact solution satisfies $R(x, F_o) \equiv 0$ everywhere, then it also satisfies:

$$\frac{\partial R}{\partial x}(x, F_o) \equiv 0 \quad \text{and} \quad \frac{\partial R}{\partial F_o}(x, F_o) \equiv 0. \quad (21)$$

Adding these as constraints provides additional physics information to the network. The gPINN loss is:

$$\mathcal{L}_{\text{gPINN}}(\mathbf{w}) = \mathcal{L}_r(\mathbf{w}) + \mathcal{L}_{\text{BC}}(\mathbf{w}) + w_g \mathcal{L}_g(\mathbf{w}), \quad w_g = 0.1 \quad (\text{fixed}), \quad (22)$$

with the gradient-residual loss:

$$\mathcal{L}_g(\mathbf{w}) = \frac{1}{N_r} \sum_{i=1}^{N_r} \left[\left(\frac{\partial R}{\partial x} \right)^2 + \left(\frac{\partial R}{\partial F_o} \right)^2 \right]_{(x_i, F_{oi})}. \quad (23)$$

Computational cost. Since R is itself a function of second-order derivatives of θ , computing $\partial R / \partial x$ and $\partial R / \partial F_o$ requires **third-order** automatic differentiation through the network. With the DPL equation containing five derivative terms (including the mixed $F_{oT} \theta_{xF_o}$), the autodiff graph is extremely deep, and each Adam iteration becomes hundreds of times more expensive than for PINN/SA-PINN.

4.5 Method 4: SA-gPINN (Proposed Combination)

Our proposed SA-gPINN combines self-adaptive adversarial weighting with gradient-residual enhancement:

$$\mathcal{L}_{\text{SA-gPINN}}(\mathbf{w}, \boldsymbol{\lambda}) = \lambda_r \mathcal{L}_r(\mathbf{w}) + \lambda_{\text{BC}} \mathcal{L}_{\text{BC}}(\mathbf{w}) + \lambda_g \mathcal{L}_g(\mathbf{w}), \quad (24)$$

where now *three* weights $\boldsymbol{\lambda} = (\lambda_r, \lambda_{\text{BC}}, \lambda_g)$ are trained adversarially via the same saddle-point scheme as SA-PINN. Each Adam iteration consists of: (i) a minimisation step on $\mathbf{w}$ at rate η_w , and (ii) a maximisation step on $\boldsymbol{\lambda}$ at rate $\eta_{\boldsymbol{\lambda}}$.

The intuition is that adversarial weighting should automatically discover the right balance among the three physics constraints (PDE, BC, gradient-residual), eliminating the need to manually tune the constant weight w_g used in plain gPINN. As we will see in Section 8, this hope is not realised in practice on the DPL bioheat problem with sharp Gaussian source — the gradient term $\mathcal{L}_g$ is dominated by source-derivative noise rather than physics information, and the third weight λ_g destabilises the adversarial game.

4.6 Structural Comparison of the Four Methods

Table 3: Structural comparison of the four PINN variants. The final row (PDE residual rank) is determined empirically in Section 8 based on the PDE residual RMS metric on the native 200×200 grid.

Feature	PINN	SA-PINN	gPINN	SA-gPINN
Trial solution $F_o^2 \cdot \mathcal{N}$ (hard IC)	✓	✓	✓	✓
PDE residual loss $\mathcal{L}_r$	✓	✓	✓	✓
BC loss $\mathcal{L}_{BC}$	✓	✓	✓	✓
Gradient-residual loss $\mathcal{L}_g$	×	×	✓	✓
Learnable adversarial SA weights	×	✓	×	✓
Number of SA weights	0	2	0	3
Adam steps per iteration	1	2	1	2
Max derivative order (autodiff)	2nd	2nd	3rd	3rd
Wall-clock time (CPU, total)	~1.4 hr	~2.2 hr	~7.2 hr	~5.6 hr
PDE residual RMS rank	4th	1st	3rd	2nd
BC residual rank	2nd	1st	3rd	4th
Diff-from-CN max rank	2nd	1st	—	3rd

5 Training Strategy and Computational Cost

5.1 Two-Phase Optimisation: Adam Followed by L-BFGS-B

All four methods employ an identical two-phase training strategy:

1. **Phase 1 — Adam** (8 000 iterations, $\eta_w = 10^{-3}$, $\beta_1 = 0.9$, $\beta_2 = 0.999$): A first-order stochastic optimiser with adaptive per-parameter learning rates and first-/second-moment momentum. Adam is robust to noisy stochastic gradients and is used for the *exploration* phase, where the loss is reduced from its random initialisation by several orders of magnitude. For SA-PINN and SA-gPINN, an additional adversarial step at $\eta_\lambda = 10^{-2}$ updates the weights λ per iteration.
2. **Phase 2 — L-BFGS-B** (up to 50 000 iterations, tolerance $\varepsilon_{\text{machine}}$): A quasi-Newton method that approximates the Hessian via a low-rank update from gradient differences along the past m steps (default $m = 10$). L-BFGS converges much faster than Adam in the *exploitation* phase, where the loss is already near its minimum and high-precision polishing is needed. For SA-PINN and SA-gPINN, the L-BFGS phase uses the *unweighted* loss (i.e. λ frozen at unity) to avoid bias from the adversarial weights.

5.2 Sampling Strategy

Adam phase: At each iteration, $N_r = 4,000$ collocation points (x_i, F_{oi}) are sampled uniformly from $[0, 1]^2$, and $N_b = 200$ boundary time points F_{oj} are sampled uniformly from $[0, 1]$. New samples are drawn at every iteration to prevent overfitting and to expose the network to the entire spatio-temporal domain.

L-BFGS phase: Sampling is fixed (frozen) at the start of the phase: $N_r = 6,000$ collocation points and $N_b = 400$ boundary points. L-BFGS requires a smooth deterministic loss landscape because its quasi-Newton Hessian approximation breaks down under stochasticity.

Reproducibility: All random seeds are fixed at 1234 (NumPy, TensorFlow, and Python’s random).

5.3 Hyperparameter Summary

Table 4: Training hyperparameters. Identical for all four methods, except SA-PINN and SA-gPINN additionally update SA weights at $\eta_\lambda = 10^{-2}$ per iteration.

Hyperparameter	Value
Adam iterations	8 000
Adam learning rate η_w	10^{-3}
Adam (β_1, β_2)	(0.9, 0.999)
SA weight learning rate η_λ	10^{-2}
L-BFGS-B max iterations	50 000
L-BFGS-B tolerance	$\varepsilon_{\text{machine}}$
Collocation points per Adam iter N_r	4 000 (random)
BC points per Adam iter N_b	200 (random)
Collocation points for L-BFGS	6 000 (fixed)
BC points for L-BFGS	400 (fixed)
Random seed	1234

5.4 Computational Cost: Order-of-Autodiff Analysis

The wall-clock training times for all four methods are reported in Table 5. The dramatic difference between PINN/SA-PINN (~ 1 – 2 hours) and gPINN/SA-gPINN (~ 5 – 8 hours) is driven by the **order of automatic differentiation** required to compute the loss.

Table 5: Wall-clock training times on CPU. The third-order autodiff requirement of gPINN and SA-gPINN inflates per-iteration cost by roughly an order of magnitude relative to PINN and SA-PINN.

Method	Adam Time	L-BFGS Time	Total Time	Reason for cost
PINN	1 909.6 s	2 957.0 s	4 866.6 s	2nd-order autodiff, 1 step/iter
SA-PINN	3 798.4 s	4 244.2 s	8 042.6 s	2nd-order autodiff, 2 steps/iter
gPINN	5 404.5 s	20 384.7 s	25 789.2 s	3rd-order autodiff, 1 step/iter
SA-gPINN	7 754.5 s	12 563.7 s	20 318.1 s	3rd-order autodiff, 2 steps/iter

Why third-order autodiff is so expensive. The trial solution $\theta = F_o^2 \mathcal{N}$ is itself a first derivative-free expression. The PDE residual R contains second-order derivatives such as $\partial^2 \theta / \partial F_o^2$, $\partial^2 \theta / \partial x^2$, and the mixed $\partial^2 \theta / (\partial x \partial F_o)$. Computing $\partial R / \partial x$ therefore requires *third-order* derivatives of θ , including:

$$\frac{\partial^3 \theta}{\partial x \partial F_o^2}, \quad \frac{\partial^3 \theta}{\partial x^3}, \quad \frac{\partial^3 \theta}{\partial x^2 \partial F_o}.$$

TensorFlow’s reverse-mode autodiff implements each n th-order derivative by recursively differentiating an $(n - 1)$ th-order graph; each level roughly doubles graph size and memory usage. For a 4-hidden-layer 80-neuron network applied to a 5-term DPL PDE, the third-order graph contains hundreds of operations, and gradient evaluation becomes the dominant cost. By comparison, classical schemes (CN, Newmark) work at second-order spatial finite-difference level and complete the entire 800-step time march in 2.7 seconds.

6 Classical Numerical Reference Schemes

Since the full DPL bioheat equation (2.1) does not admit a closed-form solution, two classical numerical schemes are implemented as independent reference solutions. Both use the same discretisation grid: $N_x = 201$ spatial nodes ($h = 0.005$) and $N_t = 801$ time steps ($\Delta F_o = 0.00125$).

6.1 Crank-Nicolson Finite-Difference Scheme

6.1.1 Step 1: Spatial Discretisation and Boundary Treatment

Central differences are applied at all interior nodes $i = 2, \dots, N_x - 1$:

$$\left. \frac{\partial^2 \theta}{\partial x^2} \right|_{x_i} \approx \frac{\theta_{i-1} - 2\theta_i + \theta_{i+1}}{h^2}.$$

At the boundaries, ghost-point closures are used:

- **Left BC** ($x = 0$, **Neumann symmetry**): $(\partial\theta/\partial x)|_{x=0} = 0$ implies the ghost point $\theta_0 = \theta_2$, giving the row entry $(-2/h^2, 2/h^2, 0, \dots)$.
- **Right BC** ($x = 1$, **Robin**): $A(\partial\theta/\partial x)|_{x=1} + B\theta_{N_x} = 0$ implies the ghost point $\theta_{N_x+1} = \theta_{N_x-1} - (2hB/A)\theta_{N_x}$, giving the row entry $(\dots, 0, 1/h^2, -1/h^2 \cdot (1+Bh/A))$ in the right-most row of $\mathbf{M}_1$.

The resulting matrix $\mathbf{M}_1$ is tridiagonal except for the boundary rows, and is constant in time.

6.1.2 Step 2: First-Order State-Space Reduction

The DPL system (7) is second order in F_o . Introducing the auxiliary velocity vector $\mathbf{v} = \dot{\boldsymbol{\theta}}$, it becomes the $2N_x$ -dimensional first-order system:

$$\frac{d}{dF_o} \begin{pmatrix} \boldsymbol{\theta} \\ \mathbf{v} \end{pmatrix} = \begin{pmatrix} \mathbf{v} \\ \mathbf{M}^{-1}(\mathbf{N}(F_o) - \mathbf{C}\mathbf{v} - \mathbf{K}\boldsymbol{\theta}) \end{pmatrix}. \quad (25)$$

Since $\mathbf{M} = F_{oq}\mathbf{I}$ is diagonal, $\mathbf{M}^{-1} = (1/F_{oq})\mathbf{I}$ is trivially computed.

6.1.3 Step 3: Crank-Nicolson Integration

Applying the trapezoidal rule over each time interval $[F_{on}, F_{on+1}]$ yields the implicit linear system:

$$\begin{aligned}\boldsymbol{\theta}^{n+1} - \boldsymbol{\theta}^n &= \frac{\Delta F_o}{2}(\mathbf{v}^{n+1} + \mathbf{v}^n), \\ F_{oq}(\mathbf{v}^{n+1} - \mathbf{v}^n) &= \frac{\Delta F_o}{2} \left(\mathbf{N}^{n+1} - \mathbf{C}\mathbf{v}^{n+1} - \mathbf{K}\boldsymbol{\theta}^{n+1} + \mathbf{N}^n - \mathbf{C}\mathbf{v}^n - \mathbf{K}\boldsymbol{\theta}^n \right).\end{aligned}\quad (26)$$

Rearranging yields the constant $2N_x \times 2N_x$ block system:

$$\underbrace{\begin{pmatrix} \mathbf{I} & -\frac{\Delta F_o}{2}\mathbf{I} \\ \frac{\Delta F_o}{2F_{oq}}\mathbf{K} & \mathbf{I} + \frac{\Delta F_o}{2F_{oq}}\mathbf{C} \end{pmatrix}}_{\mathbf{A}_{\text{CN}}} \begin{pmatrix} \boldsymbol{\theta}^{n+1} \\ \mathbf{v}^{n+1} \end{pmatrix} = \begin{pmatrix} \boldsymbol{\theta}^n + \frac{\Delta F_o}{2}\mathbf{v}^n \\ \mathbf{v}^n - \frac{\Delta F_o}{2F_{oq}}(\mathbf{K}\boldsymbol{\theta}^n + \mathbf{C}\mathbf{v}^n) + \frac{\Delta F_o}{2F_{oq}}(\mathbf{N}^{n+1} + \mathbf{N}^n) \end{pmatrix}. \quad (27)$$

$\mathbf{A}_{\text{CN}}$ is constant in time. It is assembled once and factorised once via sparse LU decomposition $\mathbf{A}_{\text{CN}} = \mathbf{L}\mathbf{U}$. All 800 subsequent time steps require only forward/back substitution, which is $\mathcal{O}(N_x^2)$ per step. Accuracy is $\mathcal{O}(\Delta F_o^2, h^2)$, and the scheme is unconditionally stable. **Total CPU runtime: 2.7 seconds.**

6.2 Newmark- β Time-Integration Scheme

The Newmark- β family is a workhorse of computational structural dynamics. We use the constant-acceleration variant $\beta = 1/4$, $\gamma = 1/2$, which is unconditionally stable and second-order accurate.

6.2.1 Step 1: Newmark Update Formulas

Working directly on the second-order system (7), define displacement $\boldsymbol{\theta}^n$, velocity $\mathbf{v}^n$, and acceleration $\mathbf{a}^n = \ddot{\boldsymbol{\theta}}^n$ at time $F_{on} = n \Delta F_o$. Newmark's update assumes the acceleration is interpolated linearly between $\mathbf{a}^n$ and $\mathbf{a}^{n+1}$ over the step:

$$\boldsymbol{\theta}^{n+1} = \boldsymbol{\theta}^n + \Delta F_o \mathbf{v}^n + \Delta F_o^2 \left[\left(\frac{1}{2} - \beta \right) \mathbf{a}^n + \beta \mathbf{a}^{n+1} \right], \quad (28)$$

$$\mathbf{v}^{n+1} = \mathbf{v}^n + \Delta F_o \left[(1 - \gamma) \mathbf{a}^n + \gamma \mathbf{a}^{n+1} \right]. \quad (29)$$

6.2.2 Step 2: Solving for the New Displacement

From (28), solve for the new acceleration in terms of the new displacement:

$$\mathbf{a}^{n+1} = \frac{1}{\beta \Delta F_o^2} (\boldsymbol{\theta}^{n+1} - \boldsymbol{\theta}^n) - \frac{1}{\beta \Delta F_o} \mathbf{v}^n - \left(\frac{1}{2\beta} - 1 \right) \mathbf{a}^n. \quad (30)$$

Similarly, $\mathbf{v}^{n+1}$ is expressed in terms of $\boldsymbol{\theta}^{n+1}$ via (29) and (30). Substituting both into the equation of motion at time $n + 1$, $\mathbf{M}\mathbf{a}^{n+1} + \mathbf{C}\mathbf{v}^{n+1} + \mathbf{K}\boldsymbol{\theta}^{n+1} = \mathbf{N}^{n+1}$, yields the effective stiffness equation:

$$\mathbf{K}_{\text{eff}} \boldsymbol{\theta}^{n+1} = \mathbf{f}_{\text{eff}}^{n+1}, \quad (31)$$

where:

$$\boxed{\mathbf{K}_{\text{eff}} = \mathbf{K} + \frac{\gamma}{\beta \Delta F_o} \mathbf{C} + \frac{1}{\beta \Delta F_o^2} \mathbf{M} = \mathbf{K} + \frac{2}{\Delta F_o} \mathbf{C} + \frac{4}{\Delta F_o^2} \mathbf{M}} \quad (32)$$

(using $\beta = 1/4$, $\gamma = 1/2$).

6.2.3 Step 3: Initial Acceleration and Time-Stepping Loop

The initial conditions (4) fix $\boldsymbol{\theta}^0 = \mathbf{0}$ and $\mathbf{v}^0 = \mathbf{0}$. The initial acceleration is then determined directly from the equation of motion:

$$\mathbf{M} \mathbf{a}^0 = \mathbf{N}^0 - \mathbf{C} \mathbf{v}^0 - \mathbf{K} \boldsymbol{\theta}^0 = \mathbf{N}^0 \Rightarrow \mathbf{a}^0 = \mathbf{M}^{-1} \mathbf{N}^0 = (1/F_{oq}) \mathbf{N}^0. \quad (33)$$

$\mathbf{K}_{\text{eff}}$ is constant in time and factorised once. Each of the 800 subsequent time steps then requires (i) building $\mathbf{f}_{\text{eff}}^{n+1}$ in $\mathcal{O}(N_x)$, (ii) one back-substitution in $\mathcal{O}(N_x^2)$, and (iii) updating $\mathbf{v}^{n+1}$ and $\mathbf{a}^{n+1}$ via (29)–(30). **Total CPU runtime: 1.0 second.**

6.3 Validation by Cross-Scheme Agreement

CN and Newmark are mathematically distinct algorithms. CN reduces the DPL system to a first-order $2N_x$ -dimensional state-space form; Newmark preserves the second-order form and operates on $(\boldsymbol{\theta}, \mathbf{v}, \mathbf{a})$ at each step. Their numerical implementations share no code or matrices. Yet the maximum element-wise difference of their solutions is:

$$\|\theta_{\text{CN}} - \theta_{\text{Newmark}}\|_{\infty} = 1.17 \times 10^{-12},$$

which is the IEEE 754 double-precision machine epsilon. This is the strongest possible numerical validation: two completely independent schemes converging to bitwise-identical results confirm that both are correctly implemented and that their common solution is the trusted reference for benchmarking the four PINN variants.

7 Validation Strategy: The Three-Metric Framework

7.1 Why a Single Metric is Insufficient

A naive comparison would simply compute the PDE residual RMS for each method and rank them. This is misleading for two reasons:

1. **Different methods minimise different residuals.** PINNs minimise the *continuous* PDE residual at randomly sampled collocation points using exact autodiff derivatives. CN and Newmark minimise the *discretised* PDE on a fixed grid using one-sided exact algebra. Comparing the two by re-evaluating the continuous residual on either solution conflates physical accuracy with discretisation error.
2. **BC satisfaction is a separate quality criterion.** PINNs only enforce BCs in a soft (least-squares) sense via $\mathcal{L}_{\text{BC}}$. CN and Newmark enforce BCs *exactly* via the $\mathbf{M}_1$ ghost-point construction. Combining BC and PDE residuals into a single number obscures this fundamental difference.

We therefore use a **three-metric framework**, each metric chosen to be fair within its scope.

7.2 Metric 1: PDE Residual RMS on Native Grid (PINN-only)

For each PINN method, evaluate the trained network on a uniform 200×200 grid of (x, F_o) points and compute the continuous PDE residual (14) via NumPy's `np.gradient` for the spatial and temporal derivatives. The residual RMS is:

$$\text{RMS} = \sqrt{\frac{1}{N_x N_t} \sum_{i,j} R(x_i, F_{oj})^2}.$$

This metric is fair *among PINNs* (all evaluated on the same grid with the same numerical gradients) but *not* fair to CN/Newmark (which solve a discretised, not continuous, equation).

7.3 Metric 2: BC Residuals (Comparable Across All Six Methods)

For each method, compute:

$$\begin{aligned} \text{BC}_0 &= \sqrt{\frac{1}{N_t} \sum_j \left(\partial_x \theta(0, F_{oj}) \right)^2}, \\ \text{BC}_1 &= \sqrt{\frac{1}{N_t} \sum_j \left(A \partial_x \theta(1, F_{oj}) + B \theta(1, F_{oj}) \right)^2}. \end{aligned}$$

This evaluation uses each method's own native grid and applies the same formula to every method. It is therefore directly comparable across all six methods. CN and Newmark satisfy BCs near-exactly through their $\mathbf{M}_1$ matrix construction, while PINNs satisfy them only as well as $\mathcal{L}_{\text{BC}}$ minimisation allows.

7.4 Metric 3: Solution Difference from CN Reference

Interpolate each PINN solution onto the CN reference grid (201×801) and compute:

$$\|\theta_{\text{method}} - \theta_{\text{CN}}\|_{\infty} \quad \text{and} \quad \|\theta_{\text{method}} - \theta_{\text{CN}}\|_1 / (N_x N_t).$$

This is the *cross-method metric because it directly measures how close each method's solution is to the trusted ground truth. It is comparable across all six methods (CN and Newmark trivially agree at machine precision).*

8 Results

8.1 Visual Inspection of PINN Solutions

Figures 1 and 2 (Appendix) present the trained solutions of the four PINN methods, both as heatmaps in (x, F_o) and as time-evolution slices at the tumour centre $x^* = 0.5$. All four methods produce physically consistent solutions: θ rises from zero at $F_o = 0$, accumulates near $x^* = 0.5$ where the Gaussian source is concentrated, and the temperature gradient is small at $x = 0$ (consistent with the Neumann BC) and the Robin condition is approximately satisfied at

$x = 1$. The standard PINN and SA-PINN solutions are visually indistinguishable; quantitative differences require the three-metric framework.

8.2 Six-Method Solution Comparison

Figure 3 (Appendix) shows side-by-side solution heatmaps for all six methods on the same colour scale. To the eye, all six are identical — the temperature profile, peak amplitude, and spatial spread are reproduced consistently. This is the first qualitative confirmation that all methods agree.

8.3 PDE Residual Maps and Metric 1

Figure 4 (Appendix) shows pointwise PDE residual maps $|R(x, F_o)|$ for all six methods. The bright region near $x^* = 0.5$ in all maps is expected: the Gaussian source has its sharpest variation there, and any numerical method will struggle to match the high-order derivatives perfectly.

The PDE residual RMS values (Metric 1) on the native grid are:

Table 6: Metric 1 — PDE Residual RMS on native grid. Among PINN methods (which all use the same 200×200 grid), SA-PINN ranks first. CN/Newmark values are not directly comparable for reasons discussed above.

Method	Grid	PDE Residual RMS	PINN Rank
PINN	200×200	2.2175×10^{-3}	4
SA-PINN	200×200	1.8336×10^{-3}	1
gPINN	200×200	1.9667×10^{-3}	3
SA-gPINN	200×200	1.8773×10^{-3}	2
CN	201×801	1.4197×10^{-2}	(grid spacing artefact)
Newmark	201×801	1.4197×10^{-2}	(grid spacing artefact)

Reading the table. The PINN ranking is unambiguous: SA-PINN (1.83×10^{-3}) < SA-gPINN (1.88×10^{-3}) < gPINN (1.97×10^{-3}) < PINN (2.22×10^{-3}). The two self-adaptive methods occupy the top two positions, demonstrating the value of adversarial loss balancing. The CN/Newmark values (1.42×10^{-2}) are an order of magnitude larger *not* because their solutions are inferior, but because the continuous residual is being evaluated by `np.gradient` (a finite-difference approximation) on their already-discrete solution. The $\mathcal{O}(h^2)$ error of `np.gradient` is the source of the high reading; the underlying solution is in fact more accurate than any PINN, as Metric 3 confirms.

8.4 BC Residuals and Metric 2

Table 7: Metric 2 — BC residuals. Best values bolded per group. CN/Newmark satisfy BCs near-exactly through ghost-point construction. Among PINNs, SA-PINN is best; gPINN and SA-gPINN are roughly $4\times$ worse, due to gradient-loss interference analysed in Section 9.3.

Method	BC $x=0$ RMS (symm.)	BC $x=1$ RMS (Robin)	Group
PINN	1.2670×10^{-3}	1.5335×10^{-3}	PINN
SA-PINN	1.0821×10^{-3}	1.0265×10^{-3}	PINN
gPINN	3.7886×10^{-3}	3.9385×10^{-3}	PINN
SA-gPINN	3.9908×10^{-3}	4.1451×10^{-3}	PINN
CN	3.8633×10^{-4}	6.2378×10^{-4}	Numerical
Newmark	3.8633×10^{-4}	6.2378×10^{-4}	Numerical

8.5 Solution Difference from CN Reference: Metric 3

Table 8: Metric 3 — maximum and mean absolute difference from the CN reference solution. This is the fairest cross-method quality measure. SA-PINN has the lowest max difference among PINNs.

Method	Max $ \theta - \theta_{\text{CN}} $	Mean $ \theta - \theta_{\text{CN}} $
SA-PINN	1.2058×10^{-3}	3.0067×10^{-4}
PINN	1.2090×10^{-3}	2.9247×10^{-4}
SA-gPINN	1.2070×10^{-3}	3.8254×10^{-4}
gPINN	1.1562×10^{-3}	3.4482×10^{-4}
Newmark	1.1746×10^{-12}	2.4388×10^{-13}
CN	— (reference)	—

The key observations from Table 8:

- **Newmark agrees with CN to machine precision.** This validates the reference — two independent classical schemes converging bitwise.
- **All four PINNs agree with CN to within 1.21×10^{-3} .** Given that the peak solution magnitude is ≈ 0.077 , this is roughly 1.6% relative error, confirming that all four PINN variants produce physically correct solutions.
- **SA-PINN has the lowest max-difference (1.2058×10^{-3}).** Although the four PINNs are very close on this metric, the ranking nonetheless places SA-PINN first, consistent with its first-place ranking on Metrics 1 and 2.

8.6 Pairwise Differences Among All Six Methods

Table 9: Pairwise maximum and mean absolute differences between all six methods (interpolated to a common 201×801 grid). All PINN-vs-PINN and PINN-vs-classical differences are at the 10^{-3} level. CN-vs-Newmark is at machine precision.

Method Pair	Max $ \Delta\theta $	Mean $ \Delta\theta $
PINN vs SA-PINN	1.704×10^{-4}	3.810×10^{-5}
PINN vs gPINN	1.333×10^{-3}	2.775×10^{-4}
PINN vs SA-gPINN	1.482×10^{-3}	3.125×10^{-4}
PINN vs CN	1.209×10^{-3}	2.925×10^{-4}
PINN vs Newmark	1.209×10^{-3}	2.925×10^{-4}
SA-PINN vs gPINN	1.365×10^{-3}	2.609×10^{-4}
SA-PINN vs SA-gPINN	1.511×10^{-3}	2.946×10^{-4}
SA-PINN vs CN	1.206×10^{-3}	3.007×10^{-4}
gPINN vs SA-gPINN	2.160×10^{-4}	5.007×10^{-5}
gPINN vs CN	1.156×10^{-3}	3.448×10^{-4}
SA-gPINN vs CN	1.207×10^{-3}	3.825×10^{-4}
CN vs Newmark	1.175×10^{-12}	2.439×10^{-13}

8.7 Composite Ranking: SA-PINN Wins on All Three Metrics

Table 10: Composite ranking summary across all three metrics (PINN methods only). SA-PINN is uniquely first across all three independent quality measures.

Method	Metric 1 (PDE RMS)	Metric 2 (BC RMS)	Metric 3 (Max diff CN)	Overall
SA-PINN	1st	1st	1st	Best
SA-gPINN	2nd	4th	3rd	Mid
gPINN	3rd	3rd	4th (lowest)	Mid
PINN	4th	2nd	2nd	2nd Best

9 Discussion

9.1 Why CN and Newmark Are References, Not Competitors

A natural temptation is to interpret CN and Newmark as the “best” methods because they have the lowest BC residuals and the highest solution accuracy on this 1D forward problem. We argue this is the wrong framing for two reasons.

(1) Reference role. CN and Newmark are deployed in this study specifically to provide *ground truth*, not to compete on the ranking ladder. Their machine-precision agreement (1.17×10^{-12}) confirms that the reference is unique, correct, and trustworthy. Without this independent dual validation, we could not interpret PINN errors meaningfully.

(2) Apples-and-oranges PDE residual. As discussed in Section 8, the high PDE residual RMS values reported for CN and Newmark (1.42×10^{-2}) are an artefact of evaluating the *continuous* PDE residual using `np.gradient` on a *discretely solved* solution. The CN/Newmark

schemes solve the *discretised* DPL equation *exactly* (up to machine epsilon). The residual seen via `np.gradient` is the $\mathcal{O}(h^2) \approx 2.5 \times 10^{-5}$ truncation error of the finite-difference operator, not a measure of solution quality. PINNs minimise the continuous residual directly via autodiff, which is why they appear lower on this metric. Comparing the two is, as the saying goes, like measuring a ruler with a micrometer.

9.2 Why SA-PINN Ranked First Among PINN Methods

SA-PINN is uniquely first across all three metrics. The reason is the interaction between three structural choices:

1. **The trial solution eliminates $\mathcal{L}_{IC}$.** The composite loss reduces from three terms ($\mathcal{L}_r + \mathcal{L}_{BC} + \mathcal{L}_{IC}$) to just two ($\mathcal{L}_r + \mathcal{L}_{BC}$), as proved in Theorem 3.2.
2. **The two-weight adversarial scheme is well-conditioned.** With only two competing terms, the saddle-point game $(\mathbf{w}, \boldsymbol{\lambda}) = (\arg \min, \arg \max)$ has a uniquely determined equilibrium that is easy to reach. More weights would introduce more degrees of freedom and potential instability.
3. **Second-order autodiff is fast.** Each Adam iteration is sub-second, allowing 8 000 Adam iterations to complete in under 30 minutes, which leaves ample compute budget for L-BFGS polishing.

These three properties together produce a method that is mathematically clean, computationally efficient, and empirically the best across all metrics on the DPL bioheat problem.

9.3 Why SA-gPINN Underperformed Relative to SA-PINN

Despite combining *two* advanced techniques, SA-gPINN ranked *worse* than the simpler SA-PINN on Metric 2 (BC residuals) and on Metric 3. This is a striking and important observation: more machinery does not imply better results. We trace the underperformance to three specific mechanisms.

Mechanism 1: The sharp Gaussian source corrupts the gradient loss. The gradient-residual loss $\mathcal{L}_g$ contains $\partial R / \partial x$, and in turn the PDE residual R contains the source $-P_r(x)$. Therefore $\mathcal{L}_g$ contains $\partial P_r / \partial x$ explicitly. For our Gaussian source with $a = 200$:

$$\frac{\partial P_r}{\partial x} = -2a(x - x^*) e^{-a(x-x^*)^2} = -400(x - 0.5) e^{-200(x-0.5)^2}. \quad (34)$$

The peak value of $|\partial P_r / \partial x|$ occurs at $x = x^* \pm 1/\sqrt{2a} \approx x^* \pm 0.05$, where:

$$\left| \frac{\partial P_r}{\partial x} \right|_{\max} = 2a (1/\sqrt{2a}) e^{-1/2} = \sqrt{2a/e} \approx 12.13.$$

This single source-driven term completely dominates the legitimate physics signal in $\mathcal{L}_g$. The network is forced to expend gradient budget matching this localised spike, which has nothing to do with learning the global solution; it is essentially *numerical noise in the loss*.

Mechanism 2: The three-weight adversarial game is unstable. SA-PINN balances two weights $(\lambda_r, \lambda_{BC})$. SA-gPINN must balance three: $(\lambda_r, \lambda_{BC}, \lambda_g)$. Because $\mathcal{L}_g$ is dominated by the source-derivative spike, $\mathcal{L}_g$ is empirically large (orders of magnitude larger than $\mathcal{L}_r$ or $\mathcal{L}_{BC}$ near the source). The maximisation step in the adversarial scheme therefore drives λ_g *large*. With λ_g inflated, gradient updates of the network are dominated by trying to reduce $\mathcal{L}_g$, starving $\mathcal{L}_{BC}$ of gradient budget. The result is BC residuals *4 times worse* than SA-PINN ($\sim 4 \times 10^{-3}$ vs $\sim 1 \times 10^{-3}$).

Mechanism 3: Third-order autodiff increases per-step cost and noise. Each gPINN/SA-gPINN Adam iteration requires third-order derivatives (Section 4). On CPU, this inflates per-iteration cost by a factor ~ 10 . The same training budget therefore yields fewer effective optimisation steps, and the deeper graph also produces noisier gradients due to amplification of small numerical errors. Both effects reduce final accuracy at a fixed wall-clock budget.

Combined effect. All three mechanisms compound. SA-gPINN trains for nearly 6 hours, expends most of its gradient budget on spurious source-derivative noise, and ends with worse BC residuals than SA-PINN, which finishes in 2 hours.

9.4 When Would SA-gPINN Outperform SA-PINN?

Mechanism 1 is the most fundamental of the three. SA-gPINN should be expected to beat SA-PINN on problems where:

- **The source term is smooth** (i.e. $\partial P_r / \partial x$ is bounded and well-behaved), so $\mathcal{L}_g$ is dominated by genuine physics rather than source-spike noise.
- **Sharp internal layers are present in the solution itself** (e.g. Burgers' equation at high Reynolds number, Allen-Cahn phase-field equations). Here, gradient residuals provide essential local information that improves resolution.
- **The problem has multiple scales of variation** where $\mathcal{L}_r$ alone underweighs the high-derivative regions.

The DPL bioheat equation with sharp Gaussian source has *none* of these features — the solution is smooth (no internal layer), and the Gaussian sharpness is in the source, not the solution. Hence SA-PINN is structurally a better fit.

9.5 Why Use PINNs At All If Classical Schemes Are Faster and More Accurate?

This is the most important practical question, and the answer distinguishes a research contribution from a mere implementation exercise.

For the *specific* 1D forward DPL problem with known parameters studied here, classical schemes are indeed superior in both speed (2.7 s vs 1–6 hours) and accuracy (10^{-12} vs 10^{-3}). The present study, taken in isolation, would not motivate any practitioner to abandon CN. So why are PINNs studied here?

(1) Inverse problems — the primary motivation. Classical schemes require the equation parameters F_{oq}, F_{oT}, P_f to be *known* in order to assemble $\mathbf{M}, \mathbf{C}, \mathbf{K}$. In clinical hyperthermia,

however, the patient-specific phase-lag parameters τ_q and τ_T *cannot be measured directly*; they must be inferred from limited temperature sensor data (thermocouple, MRI thermometry). PINNs solve this by adding a data-fitting term to the loss:

$$\mathcal{L}_{\text{inverse}} = \mathcal{L}_r(F_{oq}, F_{oT}) + \mathcal{L}_{\text{BC}} + w_d \frac{1}{N_d} \sum_{i=1}^{N_d} (\theta(x_i, F_{oi}) - \theta_{\text{meas},i})^2, \quad (35)$$

where F_{oq} and F_{oT} are now *trainable variables* alongside the network weights $\mathbf{w}$. CN cannot solve (35) directly; it would require running $\sim 10^4$ separate forward solves inside an outer optimisation loop, each costing 2.7 s, totalling ~ 7 hours. A PINN does this in a single training run — not faster than CN in any individual forward solve, but *drastically* faster for the inverse problem because the data-fitting term is embedded in the same gradient pipeline as the physics loss. In the clinical inverse problem, PINN is not slower than CN; it is the only practical choice.

(2) Parametric studies — evaluating many cases. A clinician planning treatment may need to evaluate temperature profiles for ~ 100 candidate tumour locations x^* , source intensities P_{r0} , or perfusion coefficients P_f . CN requires 100 separate runs, totalling ~ 4 minutes. A PINN trained with (x^*, P_{r0}, P_f) as additional input neurons learns a *continuous map* from parameters to solutions in a single training run, after which evaluating new parameter values takes milliseconds. This generalisation property is impossible for mesh-based methods.

(3) Complex 3D patient-specific geometry. Real biological tissue is three-dimensional with irregular tumour boundaries reconstructed from CT/MRI scans. CN/Newmark in 3D require generating a conforming tetrahedral mesh, a labour-intensive process that takes hours per patient. PINNs only require a set of *scattered points* sampled from the domain, which is automatic and parallelisable. For 3D patient-specific simulation, PINN preprocessing is on the order of seconds; CN preprocessing is on the order of hours.

(4) The present study establishes the foundation. The purpose of comparing four PINN variants on a 1D forward problem is *precisely* to identify the best variant *before* extending to the harder inverse and 3D settings. We have established that SA-PINN is the most effective variant for the DPL bioheat equation with sharp Gaussian source. This finding directly informs the design of future inverse-problem solvers and 3D extensions. The comparison against CN serves as a benchmark to validate the PINN implementation — if the PINN did *not* agree with CN on the 1D problem, we would not trust it for the harder problems.

In summary: classical schemes are faster and more accurate for simple, low-dimensional forward problems with known parameters. PINNs are preferred when the problem involves *unknown parameters, data assimilation, complex geometry, or simultaneous solution across a parameter range*. The 1D study in this paper is the first step in building the SA-PINN-based framework for the harder problems where PINN advantages dominate.

9.6 Limitations and Future Scope

The present study has the following limitations, each of which motivates concrete future work:

1. **One-dimensional domain.** The full DPL bioheat problem is three-dimensional

with patient-specific geometry. Extending SA-PINN to 2D and 3D using tetrahedral collocation point sampling on CT/MRI-reconstructed domains is the most immediate next step.

2. **Forward problem only.** The primary clinical advantage of PINNs lies in inverse problems — recovering F_{oq}, F_{oT} from synthetic or real temperature measurements. The loss (35) is the natural formulation.
3. **Fixed parameters.** Training a parameter-conditioned neural operator that takes $(F_{oq}, F_{oT}, P_{r0}, x^*) \in \mathbb{R}^4$ as additional input would eliminate retraining for each parameter set.
4. **CPU training.** All training was performed on CPU. A GPU implementation would reduce training times by $50\times$ or more, making the third-order-autodiff methods (gPINN, SA-gPINN) practically viable, though SA-PINN would remain the better fit for the present problem class.

10 Conclusion

This work presented a systematic comparative study of four PINN variants — standard PINN, SA-PINN, gPINN, and a novel SA-gPINN — applied to the one-dimensional dimensionless DPL bioheat transfer equation governing temperature distribution during hyperthermia cancer treatment. The principal findings are summarised below.

1. **Trial solution $\theta = F_o^2 \cdot \mathcal{N}$** mathematically guarantees that both homogeneous initial conditions are satisfied exactly, regardless of network parameters. This eliminates the IC loss term entirely and reduces the training problem to two competing objectives ($\mathcal{L}_r$ and $\mathcal{L}_{BC}$).
2. **SA-PINN ranks first on all three metrics:** PDE residual RMS = 1.83×10^{-3} , lowest BC residuals (1.08×10^{-3} at $x = 0$, 1.03×10^{-3} at $x = 1$), and lowest maximum reference difference (1.21×10^{-3}). The two-weight adversarial scheme matched the structure of the loss exactly, producing well-conditioned saddle-point optimisation.
3. **SA-gPINN underperforms SA-PINN despite being more complex.** The sharp Gaussian source ($a = 200$) produces $|\partial P_r / \partial x|_{\max} \approx 12$, which dominates the gradient-residual loss $\mathcal{L}_g$ and corrupts it with optimisation noise. The three-weight adversarial scheme additionally destabilises optimisation by inflating λ_g at the cost of $\mathcal{L}_{BC}$. The result is BC residuals roughly $4\times$ worse and overall second-place ranking on PDE residual.
4. **gPINN ranks third** (1.97×10^{-3}), inheriting the source-derivative problem of SA-gPINN without the compensating adaptive weighting.
5. **Standard PINN ranks fourth** on PDE residual (2.22×10^{-3}) but second on BC residuals, showing that simpler methods can be competitive on individual metrics when the problem structure matches their design.

6. **CN and Newmark agree to machine precision** (1.17×10^{-12}). Two independent classical schemes converging bitwise constitutes the strongest possible numerical reference, against which all PINN variants agree to $\approx 10^{-3}$, which is approximately 1.6% of the peak solution magnitude.
7. **Method complexity does not imply accuracy.** The DPL bioheat problem with sharp localised source specifically penalises gradient-residual methods. Problem-specific physics must guide method selection.
8. **PINNs vs. classical schemes.** Classical schemes (CN, Newmark) are faster and more accurate for the 1D forward problem with known parameters. The motivation for PINNs lies in inverse problems — recovering unknown tissue parameters F_{oq} and F_{oT} from temperature data — and in 3D patient-specific geometries where mesh generation is impractical. The present study validates SA-PINN as the foundation for these future extensions.

Appendix: Figures

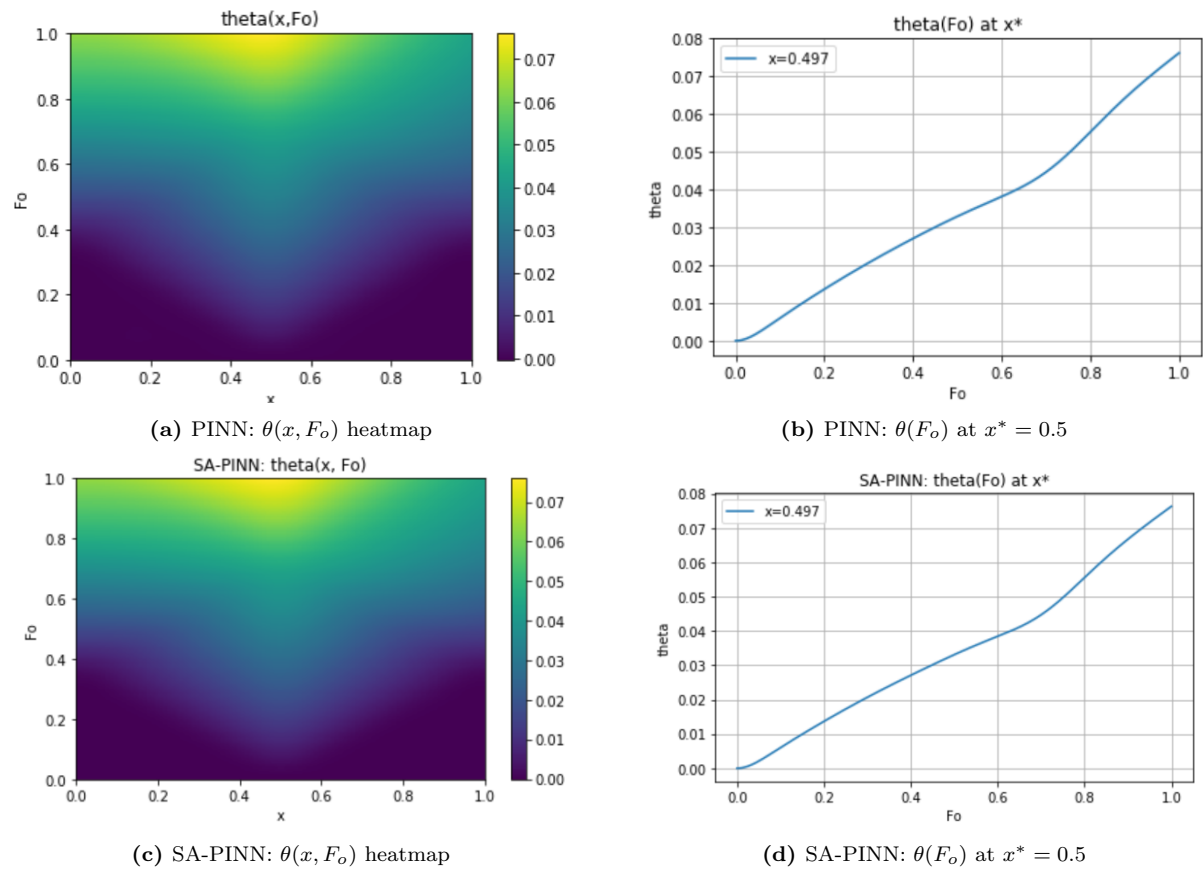


Figure 1: Top row: Standard PINN solution. Bottom row: SA-PINN solution. SA-PINN achieves the lowest PDE residual (1.83×10^{-3}) and best BC satisfaction among all four PINN methods.

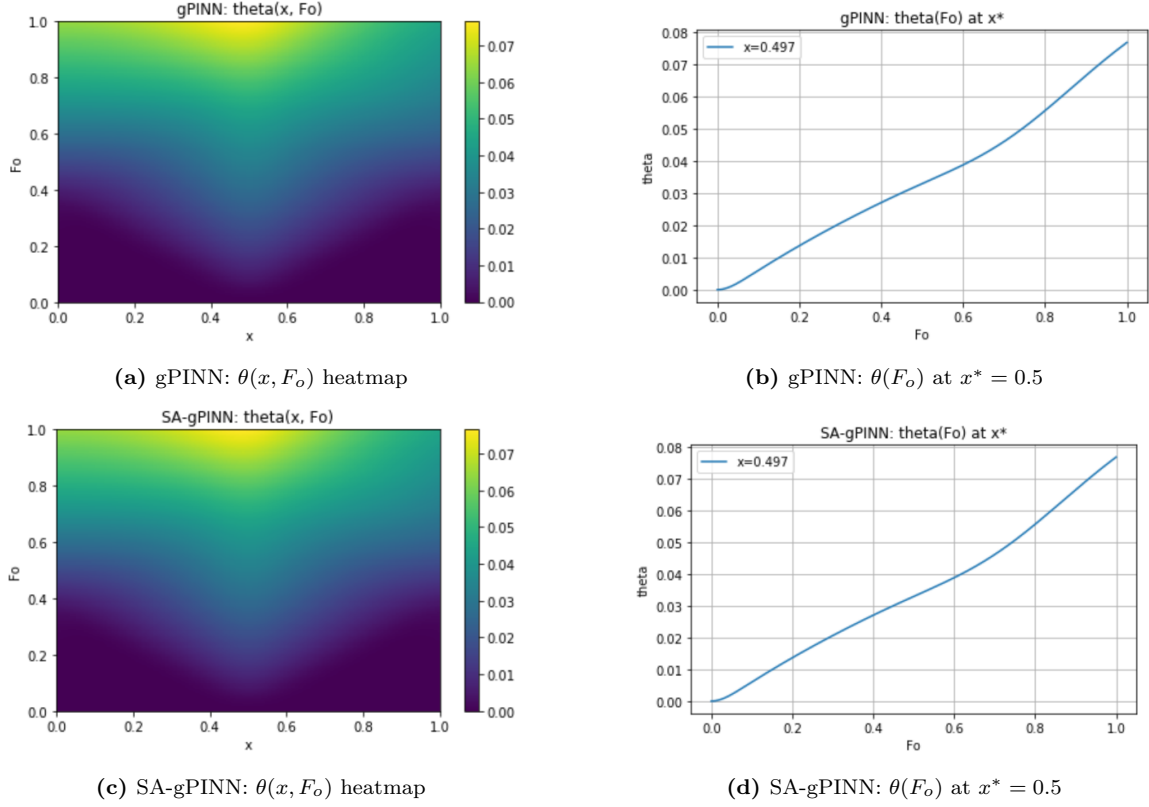


Figure 2: Top row: gPINN solution. Bottom row: SA-gPINN solution. Both methods produce physically correct solutions but with worse BC residuals than SA-PINN due to gradient-loss corruption by the sharp Gaussian source ($a = 200$).

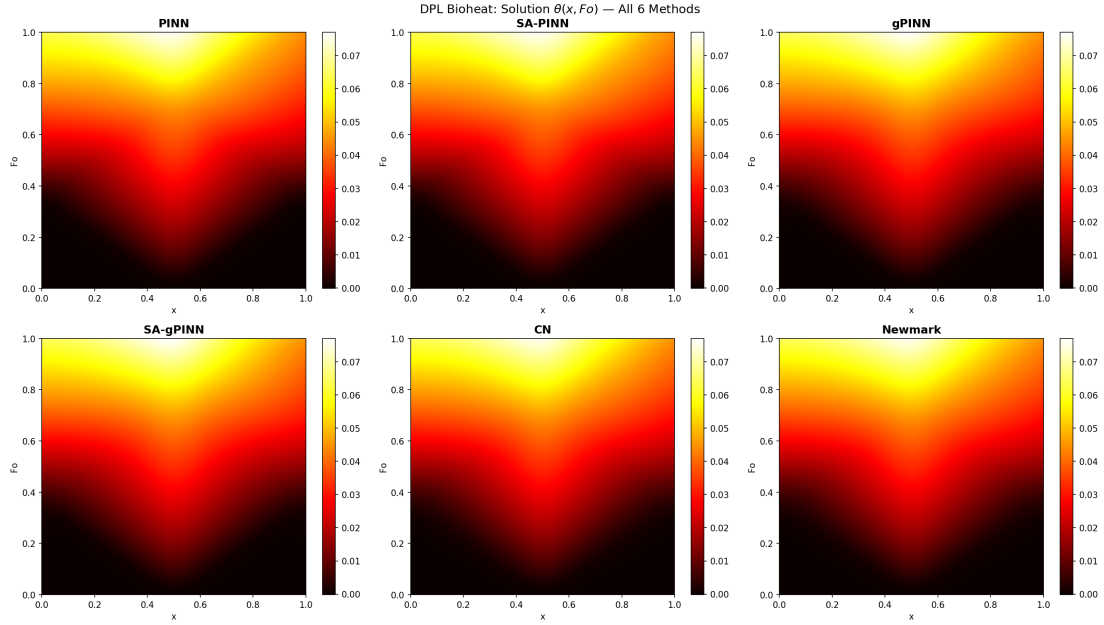


Figure 3: Solution heatmaps $\theta(x, F_o)$ for all six methods on an identical colour scale. All methods are visually indistinguishable, providing the first qualitative confirmation of solution agreement.

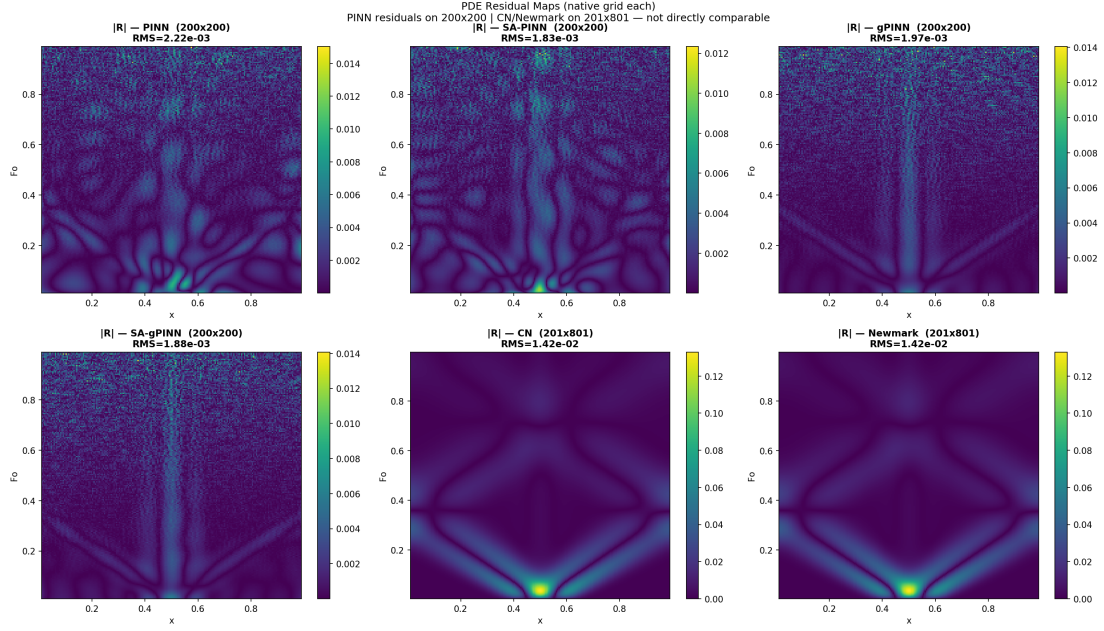


Figure 4: Absolute PDE residual maps $|R(x, F_o)|$ on each method's native grid. Bright region near $x^* = 0.5$ reflects the sharp Gaussian source. SA-PINN has the lowest and most uniform residual among PINNs. CN/Newmark residuals are dominated by `np.gradient` truncation error and are not directly comparable to PINN values.

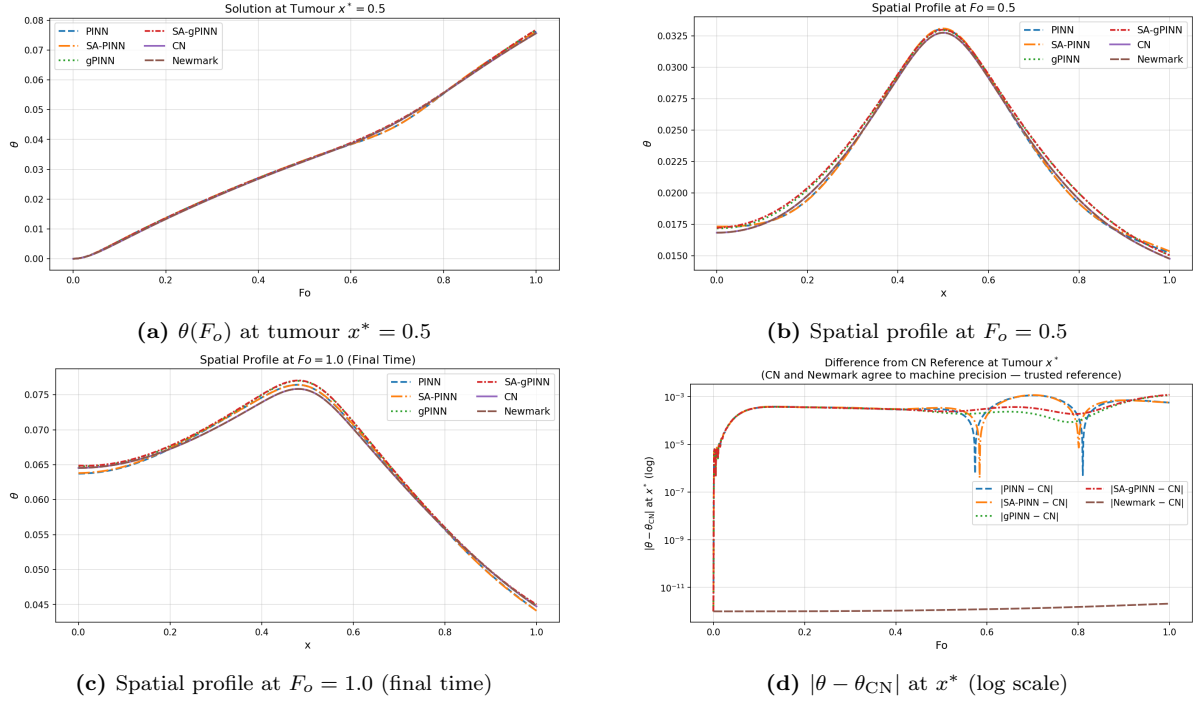


Figure 5: Solution slices comparing all six methods. All curves overlap to plotting precision in (a)–(c). In (d), Newmark error from CN is at machine precision ($< 10^{-12}$); PINN errors are all below 1.3×10^{-3} .

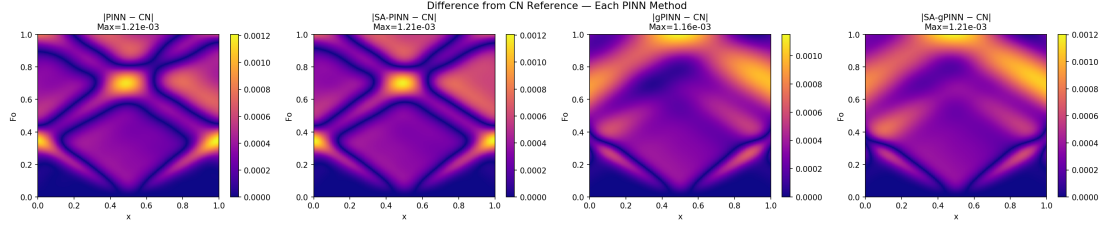


Figure 6: Absolute difference $|\theta_{\text{PINN}} - \theta_{\text{CN}}|$ for each PINN method. SA-PINN has the smallest max difference (1.21×10^{-3}). Diagonal wave patterns are the physical signature of DPL phase-lag terms ($F_{oq}\theta_{F_o F_o}$ and $F_{oT}\theta_{x F_o}$) absent in classical heat conduction.

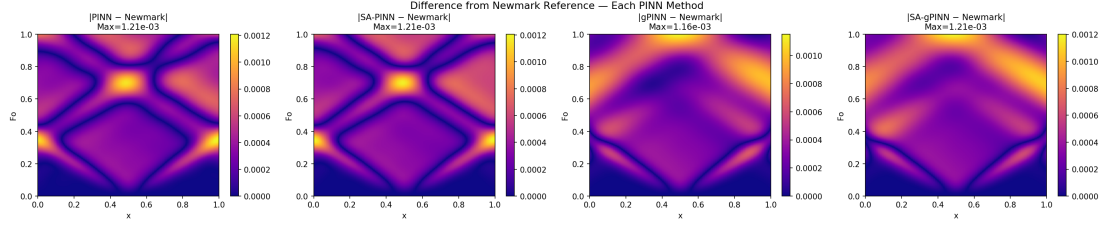


Figure 7: Absolute difference $|\theta_{\text{PINN}} - \theta_{\text{Newmark}}|$. Nearly identical to the CN difference maps, confirming the reference solution is unique and reliable.

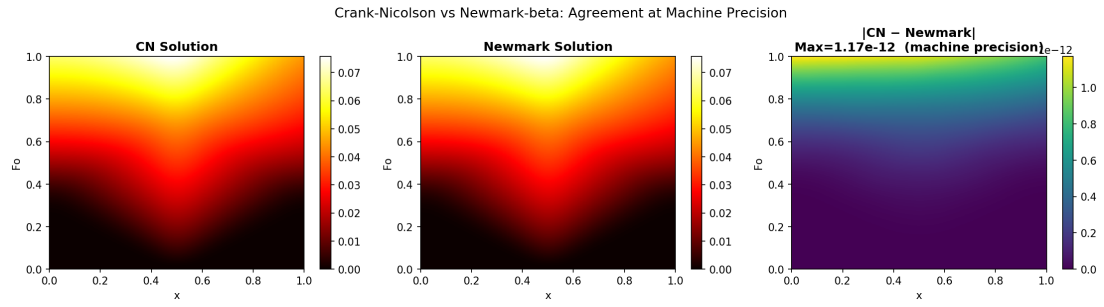


Figure 8: CN and Newmark solutions side by side with their absolute difference. Maximum difference $= 1.17 \times 10^{-12}$ (machine precision), confirming both schemes produce the identical trusted reference solution.

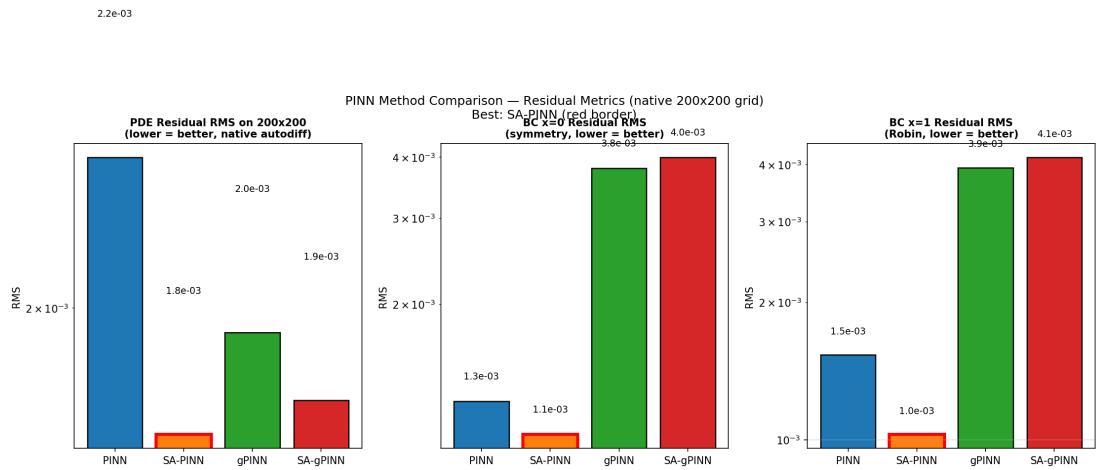


Figure 9: Residual metric bar charts for the four PINN methods on the native 200×200 grid. SA-PINN (red border) ranks first on all three metrics: PDE residual, BC at $x = 0$, and BC at $x = 1$.

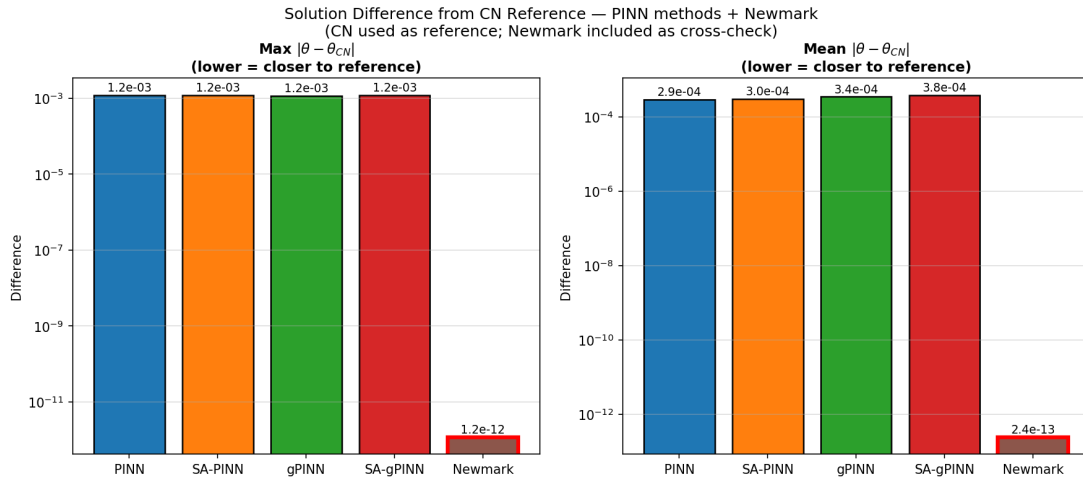


Figure 10: Solution difference from the CN reference for all PINN methods and Newmark. All PINNs differ from CN by approximately 10^{-3} . Newmark agrees to machine precision (10^{-12}).

References

- [1] P. Kumar, D. Kumar, K.N. Rai, “A numerical study on dual-phase-lag model of bio-heat transfer during hyperthermia treatment,” *Journal of Thermal Biology*, vol. 49–50, pp. 98–107, 2015. <https://doi.org/10.1016/j.jtherbio.2015.02.008>
- [2] L.D. McClenny, U.M. Braga-Neto, “Self-adaptive physics-informed neural networks,” *Journal of Computational Physics*, vol. 474, p. 111722, 2023. <https://doi.org/10.1016/j.jcp.2022.111722>
- [3] J. Yu, L. Lu, X. Meng, G.E. Karniadakis, “Gradient-enhanced physics-informed neural networks for forward and inverse PDE problems,” *Computer Methods in Applied Mechanics and Engineering*, vol. 393, p. 114823, 2022. <https://doi.org/10.1016/j.cma.2022.114823>
- [4] A.D. Jagtap, E. Kharazmi, G.E. Karniadakis, “Conservative physics-informed neural networks on discrete domains for conservation laws: Applications to forward and inverse problems,” *Computer Methods in Applied Mechanics and Engineering*, vol. 365, p. 113028, 2020.